



Casa abierta al tiempo

Universidad Autónoma Metropolitana
Azcapotzalco

DIVISIÓN DE CIENCIAS BÁSICAS E INGENIERÍA

Sistema de Asignación Automática de Grupos

Reporte Técnico

Proyecto CAT26P / 26O

Dr. César Simón López-Monsalvo
División de Ciencias Básicas e Ingeniería
cslm@azc.uam.mx

Versión 2.0
9 de junio de 2026

Índice

1. Descripción general del sistema	3
1.1. Propiedades fundamentales	3
1.2. Requisitos	3
1.3. Estructura de archivos	3
2. Modelo matemático del problema	4
2.1. Conjuntos y estructuras fundamentales	4
2.2. Condición de traslape horario	4
2.3. Función de asignación y conjunto de restricciones	5
2.4. Objetivo de optimización	6
2.5. Complejidad algorítmica por fase	6
2.6. Estructura horaria de UEA 1100037 y límite de capacidad	8
2.7. Verificación computacional y extensión al carril de 13:00	10
3. Modelos de datos	10
3.1. TimeSlot — franja horaria	11
3.2. Group — grupo académico	11
3.3. Student — estudiante	11
4. Librerías y dependencias	12
4.1. pandas	12
4.2. openpyxl	12
4.3. Biblioteca estándar de Python	14
5. Agentes del sistema	14
5.1. Agente 0 — Coordinator (orquestador)	15
5.1.1. Responsabilidad	15
5.1.2. Interfaz pública	15
5.1.3. Flujo de invocación	15
5.2. Agente 1 — ProposalVerifier	15
5.2.1. Responsabilidad	15
5.2.2. Funciones implementadas	16
5.2.3. Algoritmo de detección intra-CAT	16
5.2.4. Salida	16
5.3. Agente 2 — DataProcessor	16
5.3.1. Responsabilidad	16
5.3.2. Funciones implementadas	17
5.3.3. Regla de capacidad	17
5.3.4. Detección dinámica de columnas de UEA	17
5.3.5. Salida	17
5.4. Agente 3 — Scheduler (motor de asignación)	17
5.4.1. Responsabilidad	17
5.4.2. Estructuras de control internas	17
5.4.3. Intuición del algoritmo	17
5.4.4. Pseudocódigo del algoritmo voraz	18
5.4.5. Funciones implementadas	19
5.4.6. Complejidad	19
5.4.7. Salida	19

5.5. Agente 4 — Reporter	19
5.5.1. Responsabilidad	19
5.5.2. Funciones implementadas	19
5.5.3. Algoritmo de volcado	19
5.5.4. Salidas	20
5.6. Agente 5 — OverlapVerifier (auditoría post-asignación)	20
5.6.1. Responsabilidad	20
5.6.2. Funciones implementadas	20
5.6.3. Auditoría A — Traslapes por estudiante	20
5.6.4. Auditoría B — Agrupamiento por plan (UEA 1100037)	21
5.6.5. Salida	21
6. Herramientas independientes	21
6.1. verify_overlaps.py	21
6.2. analyze_morning_capacity.py	21
7. Punto de entrada — main.py	22
8. Archivos de entrada y salida	22
8.1. Entradas requeridas	22
8.2. Salidas producidas	22
9. Incorporar un nuevo ciclo trimestral	22
10. Resumen de responsabilidades	23
11. Conclusiones	23

1 Descripción general del sistema

Cada trimestre, la División de Ciencias Básicas e Ingeniería asigna a varios cientos de estudiantes de nuevo ingreso a los grupos de sus unidades de enseñanza-aprendizaje (UEA). La versión manual del proceso exige verificar, para cada estudiante, que ninguna de sus seis materias genere un conflicto de horario con otra, que todos los grupos pertenezcan al carril CAT correcto, que los cupos no se excedan y que el alumnado de un mismo plan quede agrupado cohesionadamente — todo simultáneamente. Con $N = 674$ estudiantes procesados en el ciclo 26-Primavera y $G_{\text{máx}} = 26$ grupos disponibles por UEA, el número de combinaciones posibles para un solo estudiante —antes de aplicar cualquier restricción— es

$$G_{\text{máx}}^{U_{\text{máx}}} = 26^6 = 308,915,776 \approx 3.1 \times 10^8.$$

Para los 674 estudiantes en simultáneo, el espacio de búsqueda completo alcanza $(26^6)^{674} = 26^{4044} \approx 10^{5722}$ configuraciones posibles; la revisión exhaustiva es, en la práctica, imposible.

El sistema CAT26P es una aplicación en Python diseñada para la asignación automática de estudiantes de nuevo ingreso a los grupos académicos (UEA) de la División de Ciencias Básicas e Ingeniería de la UAM-Azcapotzalco. Su arquitectura sigue el patrón *multi-agente orientado a flujo de trabajo* [8]: un orquestador central (Coordinator) invoca secuencialmente cinco agentes especializados, cada uno responsable de una fase bien delimitada.

1.1 Propiedades fundamentales

El sistema tiene seis propiedades operativas que lo definen. Primero, garantiza **cero empalmes horarios**: el motor verifica exhaustivamente que ningún par de grupos asignados a un mismo estudiante comparta franja horaria. Segundo, la **carga máxima es de seis UEA por estudiante**, correspondiente al plan de primer ingreso del CBI. Tercero, el sistema aplica **prioridad matutina para alumnas**: los perfiles con `gen == 'F'` se procesan primero y reciben preferencia en grupos cuyas franjas inician antes de las 14:00 h. Cuarto, **solo se asignan grupos CAT**: grupos con nomenclatura CPRO, CTG u otra se descartan automáticamente. Quinto, se impone **agrupamiento por plan** para la UEA 1100037: todo el alumnado de un mismo plan comparte el mismo carril CAT. Sexto, la ejecución termina en **menos de un segundo** para el padrón completo.

1.2 Requisitos

El sistema requiere Python 3.10 o superior (invocado como `python3`) con los paquetes `pandas` y `openpyxl` instalados.

Advertencia No utilizar el comando `python` (sin el 3); puede apuntar a una versión incompatible o no existir en el sistema.

1.3 Estructura de archivos

```
CAT26P/
|-- main.py                # Punto de entrada: flujo completo
|-- verify_overlaps.py    # Auditoria independiente de traslapes
|-- analyze_morning_capacity.py # Analisis de capacidad matutina
|-- agents/
|   |-- coordinator.py    # Orquestador -- Fase 0
|   |-- proposal_verifier.py # Fase 1: traslapes intra-CAT
|   |-- data_processor.py  # Fase 2: ingesta y normalizacion
|   |-- scheduler.py       # Fase 3: motor de asignacion
|   |-- reporter.py        # Fase 4: asignaciones y metricas
|   |-- overlap_verifier.py # Fase 5: auditoria post-asignacion
|   '-- models.py         # Modelos de datos
```

-- 26P/	# Ciclo 26-Primavera
' -- 260/	# Ciclo 26-0tono

2 Modelo matemático del problema

El problema de asignación de grupos se formula como un *problema de satisfacción de restricciones* (*Constraint Satisfaction Problem*, CSP) [5, 2]. En un CSP se busca asignar valores a un conjunto de variables de modo que se satisfaga simultáneamente un conjunto de restricciones. En este caso, las *variables* son los pares (estudiante, UEA), los *valores* son los grupos disponibles en el catálogo, y las *restricciones* codifican las reglas operativas del División: cupos, carriles CAT, exclusión de grupos de complemento, agrupamiento por plan de estudios y ausencia de empalmes horarios.

La dificultad del problema no reside en la complejidad de cada restricción por separado — cada una se verifica en tiempo constante o lineal — sino en su interacción sobre un espacio de búsqueda de tamaño exponencial [6, 3]. La función objetivo es maximizar la cobertura: el número total de pares (estudiante, UEA) efectivamente asignados.

2.1 Conjuntos y estructuras fundamentales

El modelo descansa sobre cuatro conjuntos que delimitan el universo del problema. El primero, $\mathcal{S} = \{s_1, \dots, s_N\}$, reúne a los N estudiantes de nuevo ingreso que el sistema debe atender. El segundo, $\mathcal{U}$, agrupa todas las Unidades de Enseñanza-Aprendizaje disponibles, cada una identificada por su clave numérica (1100037, 1111078, ...). El tercero, $\mathcal{D} = \{\text{Lunes, Martes, Miércoles, Jueves, Viernes}\}$, representa los cinco días lectivos de la semana escolar. El cuarto, $\mathcal{G}$, reúne todos los grupos académicos que el catálogo del ciclo pone a disposición.

Cada grupo $g \in \mathcal{G}$ queda completamente especificado por una cuádrupla

$$g = (\text{clave}(g), \text{id}(g), \text{cap}(g), \Sigma(g)), \quad (1)$$

donde $\text{clave}(g) \in \mathcal{U}$ indica la UEA que el grupo imparte, $\text{id}(g) \in \{\text{CAT01}, \dots, \text{CAT86}\}$ es el carril CAT al que pertenece, y $\text{cap}(g) \in \{36, 45\}$ es el cupo máximo. El cuarto componente, $\Sigma(g)$, es el conjunto de franjas horarias del grupo. Denotando por $\mathbb{T} = \{00:00, 00:01, \dots, 23:59\}$ el eje temporal discreto en formato HH:MM, cada franja es un elemento de

$$\Sigma(g) \subseteq \mathcal{D} \times \mathbb{T}^2, \quad (2)$$

esto es, un par (día, intervalo temporal) que localiza al grupo dentro de la semana escolar.

Cada estudiante $s \in \mathcal{S}$, por su parte, queda especificado por una cuádrupla análoga,

$$s = (\text{mat}(s), \pi(s), \gamma(s), R(s)), \quad (3)$$

donde $\text{mat}(s)$ es su número de matrícula, $\pi(s)$ es la clave del plan de estudios en el que fue admitido, $\gamma(s) \in \{F, M\}$ es el género de inscripción —relevante para el ordenamiento prioritario de la asignación—, y $R(s) \subseteq \mathcal{U}$ con $|R(s)| \leq 6$ es el conjunto de UEA que el estudiante debe cursar en el ciclo.

2.2 Condición de traslape horario

Dos franjas $\sigma_A = (d_A, t_A^i, t_A^f)$ y $\sigma_B = (d_B, t_B^i, t_B^f)$ se *traslapan* si y solo si sus intervalos temporales se intersectan en el mismo día:

$$\sigma_A \cap \sigma_B \neq \emptyset \iff d_A = d_B \wedge t_A^i < t_B^f \wedge t_B^i < t_A^f. \quad (4)$$

Dos grupos g y g' son *compatibles* si y solo si ningún par de sus franjas se traslapa:

$$\text{compat}(g, g') = 1 \iff \nexists (\sigma, \sigma') \in \Sigma(g) \times \Sigma(g') : \sigma \cap \sigma' \neq \emptyset. \quad (5)$$

La compatibilidad es la condición necesaria y suficiente para asignar g y g' simultáneamente al mismo estudiante.

Nota La implementación aprovecha que las cadenas 'HH:MM' con formato uniforme conservan el orden cronológico bajo comparación lexicográfica estándar de Python. La condición $t_A^i < t_B^f \wedge t_B^i < t_A^f$ se evalúa directamente sobre strings sin conversión numérica.

2.3 Función de asignación y conjunto de restricciones

La asignación es una función parcial

$$\phi : \mathcal{S} \times \mathcal{U} \rightharpoonup \mathcal{G}, \quad (6)$$

definida para el par (s, u) únicamente cuando existe un grupo factible. El sistema impone cinco restricciones operativas, $\mathcal{C} = \{R1, \dots, R5\}$, cada una de las cuales captura una regla distinta del CAT.

R1: Cupo. Ningún grupo puede recibir más estudiantes que su cupo máximo. La restricción de cupo establece que

$$|\{s \in \mathcal{S} : \phi(s, u) = g\}| \leq \text{cap}(g) \quad \forall g \in \mathcal{G}, \quad (7)$$

donde el lado izquierdo de la desigualdad (7) cuenta cuántos estudiantes han sido asignados al grupo g . Esta restricción es la más activa durante la ejecución: en cuanto un grupo se llena, el algoritmo busca un grupo alternativo sin retroceder sobre asignaciones previas.

R2: Grupo CAT. La asignación solo puede recaer en grupos del sistema CAT, es decir, grupos cuyo identificador comienza con el prefijo CAT:

$$\phi(s, u) \in \mathcal{G}_{\text{CAT}} := \{g \in \mathcal{G} : \text{id}(g) \text{ comienza con CAT}\}. \quad (8)$$

La restricción de pertenencia (8) descarta automáticamente los grupos de servicio social, intercambio y educación continua que coexisten en el catálogo pero están fuera del alcance del sistema.

R3: Exclusión de carriles de complemento. Los carriles CAT12–CAT18 son laboratorios adicionales creados para absorber al alumnado que no cupo en los carriles principales, cubriendo la diferencia entre el cupo de los grupos regulares (45 estudiantes) y el de los grupos de laboratorio (36 estudiantes). Ciertas UEA carecen de sección en estos carriles; para el conjunto $\mathcal{U}_{\text{exc}} = \{1111078, 1112013, 1112042, 1113084, 1100037\}$ la restricción de exclusión es

$$u \in \mathcal{U}_{\text{exc}} \Rightarrow \text{id}(\phi(s, u)) \notin \{\text{CAT12}, \dots, \text{CAT18}\}, \quad (9)$$

es decir, si la UEA pertenece al conjunto de exclusión, la restricción (9) impide asignar un grupo en los carriles de complemento.

R4: Agrupamiento por plan (UEA 1100037). Esta restricción garantiza que todo el alumnado de un mismo plan de estudios π comparta el mismo carril para la UEA de integración. El sistema admite a lo sumo $\kappa(\pi) \in \{1, 2, 3\}$ carriles distintos por plan, donde el número de carriles asignados escala con el tamaño del plan $n(\pi) = |\{s \in \mathcal{S} : \pi(s) = \pi\}|$ según

$$\kappa(\pi) = \begin{cases} 1 & \text{si } n(\pi) \leq 110, \\ 2 & \text{si } 110 < n(\pi) \leq 220, \\ 3 & \text{si } n(\pi) > 220. \end{cases} \quad (10)$$

El agrupamiento se formaliza en dos condiciones complementarias:

$$\pi(s) = \pi(s') \Rightarrow \phi(s, 1100037) = \phi(s', 1100037), \quad (11)$$

$$|\{\phi(s, 1100037) : \pi(s) = \pi\}| \leq \kappa(\pi). \quad (12)$$

La condición de igualdad (11) impone que dos estudiantes del mismo plan reciban el mismo carril; la condición de acotamiento (12) limita a $\kappa(\pi)$ el número de carriles distintos que un plan puede ocupar.

R5: Cero empalmes. Ningún par de grupos asignados al mismo estudiante puede compartir una franja horaria. La restricción de compatibilidad requiere que

$$u_i \neq u_j \Rightarrow \text{compat}(\phi(s, u_i), \phi(s, u_j)) = 1 \quad \forall u_i, u_j \in R(s), \quad (13)$$

donde *compat* es la función de compatibilidad horaria definida en (5). La restricción (13) garantiza que el horario resultante para cada estudiante esté libre de conflictos.

2.4 Objetivo de optimización

El sistema maximiza la cobertura total: el número de pares (s, u) para los que $\phi(s, u)$ queda definida. Denotando por $\mathbf{1}[P]$ el indicador que vale 1 si la proposición P es verdadera y 0 en caso contrario, el problema de optimización toma la forma

$$\max_{\phi} \sum_{s \in \mathcal{S}} \sum_{u \in R(s)} \mathbf{1}[\phi(s, u) \text{ definido}] \quad \text{sujeto a } \mathcal{C}. \quad (14)$$

El sistema resuelve este problema mediante un *algoritmo voraz* con ordenamiento prioritario de alumnas [1]. El algoritmo produce soluciones de alta cobertura en tiempo $O(N)$.

Nota ¿Por qué no búsqueda exhaustiva? Con los datos reales del ciclo 26-Primavera ($N = 674$, $G_{\text{máx}} = 26$, $U_{\text{máx}} = 6$), el tamaño del espacio de búsqueda se obtiene así. Para un estudiante aislado, cada una de sus 6 UEA puede asignarse a cualquiera de los 26 grupos disponibles; el número de combinaciones es

$$26^6 = 26 \times 26 \times 26 \times 26 \times 26 \times 26 = 308,915,776.$$

Esto es más de 3×10^8 configuraciones *por estudiante* antes de verificar una sola restricción. Para los $N = 674$ estudiantes tratados como variables independientes, el espacio conjunto crece como el producto de los espacios individuales:

$$(26^6)^{674} = 26^{4044} \approx 10^{5722}.$$

A modo de referencia, el número estimado de átomos en el universo observable es $\approx 10^{80}$, un número 10^{5642} veces más pequeño que el espacio de búsqueda del problema. El algoritmo voraz sacrifica la garantía de optimalidad global por una complejidad polinomial $O(N)$, alcanzando en la práctica coberturas superiores al 99 % porque el catálogo CAT es suficientemente rico respecto al número de restricciones activas por estudiante.

2.5 Complejidad algorítmica por fase

Para medir el costo computacional del sistema conviene introducir cuatro parámetros. $N = |\mathcal{S}|$ es el número de estudiantes procesados; $U_{\text{máx}} = \max_s |R(s)| \leq 6$ es el número máximo de UEA que un estudiante puede requerir; $G_{\text{máx}}$ es el número máximo de grupos disponibles por UEA en el catálogo; y $S_{\text{máx}}$ es el número máximo de franjas horarias de un grupo. La tabla 2.5 resume la operación dominante y la complejidad en el peor caso para cada fase.

Fase	Operación dominante	Complejidad
1	Detección de pares en conflicto intra-CAT	$O\left(\sum_k \binom{ \mathcal{G}^k }{2} S_{\text{máx}}^2\right)$
2	Lectura del padrón y catálogo	$O(N + \mathcal{G})$
3	Asignación voraz (peor caso)	$O(N \cdot U_{\text{máx}} \cdot G_{\text{máx}} \cdot U_{\text{máx}} \cdot S_{\text{máx}})$
5	Auditoría A: pares de UEA por estudiante	$O(N \cdot U_{\text{máx}}^2 \cdot S_{\text{máx}}^2)$

Fase 1 — Detección de conflictos intra-CAT. El agente `ProposalVerifier` recorre el catálogo separando los grupos por carril CAT. Para cada carril k , examina todos los pares de grupos $\{g, g'\} \subseteq \mathcal{G}^k$ y comprueba si sus franjas horarias se traslapan. El número de pares en el carril k es $\binom{|\mathcal{G}^k|}{2}$; cada comparación requiere revisar hasta $S_{\text{máx}}^2$ pares de franjas aplicando la condición de traslape (4). Sumando sobre todos los carriles, la complejidad de la Fase 1 es

$$O\left(\sum_k \binom{|\mathcal{G}^k|}{2} S_{\text{máx}}^2\right). \quad (15)$$

La suma en (15) crece cuadráticamente con el tamaño de cada carril, no con el catálogo completo. En el ciclo 26-Primavera hay $K = 23$ carriles CAT activos, cada uno con a lo sumo $|\mathcal{G}^k| = U_{\text{máx}} = 6$ grupos (uno por UEA); la cota superior resulta

$$K \cdot \binom{U_{\text{máx}}}{2} \cdot S_{\text{máx}}^2 = 23 \times 15 \times 25 = 8,625,$$

y el valor exacto —usando los tamaños reales de cada carril— es 3,600 comparaciones de franjas. La Fase 1 es negligible frente a las $\sim 3 \times 10^6$ operaciones de la Fase 3 (16).

Fase 2 — Ingesta de datos. El agente `DataProcessor` lee el padrón de N estudiantes y el catálogo de $|\mathcal{G}|$ grupos construyendo una tabla de búsqueda indexada por clave de UEA. Cada registro se procesa exactamente una vez, por lo que la complejidad es lineal: $O(N + |\mathcal{G}|)$.

Fase 3 — Asignación voraz. Esta es la fase central y la que determina la complejidad global del sistema. El agente `Scheduler` itera sobre los N estudiantes en orden de prioridad. Para cada estudiante, recorre sus $U_{\text{máx}}$ UEA requeridas y, para cada UEA, examina en orden los $G_{\text{máx}}$ grupos disponibles en el catálogo hasta encontrar el primero factible. Verificar que un grupo g es factible para el estudiante s requiere comprobar que g es compatible con cada uno de los grupos ya asignados a s ; en el peor caso hay $U_{\text{máx}}$ grupos asignados, y cada comparación de compatibilidad cuesta $O(S_{\text{máx}})$ por la condición (5). La complejidad total de la Fase 3 en el peor caso es

$$O(N \cdot U_{\text{máx}} \cdot G_{\text{máx}} \cdot U_{\text{máx}} \cdot S_{\text{máx}}) = O(N \cdot U_{\text{máx}}^2 \cdot G_{\text{máx}} \cdot S_{\text{máx}}). \quad (16)$$

Dado que el algoritmo no retrocede, el factor $G_{\text{máx}}$ aparece solo una vez: si ningún grupo es factible para una UEA, el sistema registra la asignación parcial y continúa sin reexaminar decisiones anteriores. Con los valores del ciclo 26-Primavera ($N = 674$, $U_{\text{máx}} = 6$, $G_{\text{máx}} = 26$, $S_{\text{máx}} = 5$), la expresión de Fase 3 (16) da

$$674 \times 36 \times 26 \times 5 = 3,157,680$$

operaciones elementales en el peor caso.

Fase 5 — Auditoría post-asignación. El agente `OverlapVerifier` verifica que la solución final sea libre de empalmes. Para cada estudiante revisa todos los pares de UEA asignadas, que son a lo

sumo $(\frac{U_{\max}}{2}) \in O(U_{\max}^2)$, y para cada par de grupos compara sus $S_{\max}^2$ pares de franjas aplicando nuevamente la condición (4). La complejidad de la auditoría es

$$O(N \cdot U_{\max}^2 \cdot S_{\max}^2). \quad (17)$$

Esta expresión es independiente de $G_{\max}$ porque en la Fase 5 ya no se recorre el catálogo: solo se examinan los grupos efectivamente asignados. Con los mismos valores del ciclo, la expresión de Fase 5 (17) da

$$674 \times 36 \times 25 = 606,600$$

operaciones, aproximadamente el 19 % del costo de la Fase 3.

El número total de operaciones elementales — sumando las cuatro fases — es del orden de 3.8×10^6 , consistente con la ejecución observada de menos de un segundo.

2.6 Estructura horaria de UEA 1100037 y límite de capacidad

Las cinco UEA de primer trimestre distintas de 1100037 distribuyen sus sesiones a lo largo del día: las cuatro UEA académicas (Física, Complementos de Matemáticas, Introducción al Cálculo y Estructura Atómica) ofrecen secciones de 90 min en turno matutino y vespertino (CAT81 en adelante), mientras que el Laboratorio de Reacciones Químicas (1113085, cupo de 36 plazas por sección) dispone de bloques de tres horas en cuatro franjas: 07:00–10:00, 10:00–13:00, 15:00–18:00 y 18:00–21:00. La UEA 1100037 (Introducción a la Ingeniería) tiene una única sesión semanal de 90 min de 13:00 a 14:30 y se asigna por bloque de plan mediante la regla $\kappa(\pi)$ (10).

Ninguna de las cuatro franjas de 1113085 cae en el intervalo $[13:00, 14:30]$ de 1100037, por lo que para todo par de grupos (g, g') con $g \in \mathcal{G}_{1100037}$ y $g' \in \mathcal{G}_{1113085}$,

$$\tau(g, g') = \emptyset, \quad (18)$$

y R5 queda satisfecha vacuamente para toda combinación que incluya un grupo de cada una de estas dos UEA. La asignación de bloques de plan a 1100037 es así completamente independiente de la asignación al laboratorio 1113085 y, por R4, puede resolverse una vez concluida la Fase 1. En las cuatro UEA académicas, un pequeño número de secciones vespertinas comparte la franja 13:00–14:30 con ciertos grupos de 1100037; en esos casos R5 actúa algorítmicamente durante la Fase 1 y descarta la combinación conflictiva.

El máximo de nuevo ingreso que el ciclo puede absorber queda determinado por la *capacidad bruta total* de cada UEA,

$$C_0(u) = \sum_{g \in \mathcal{G}_u} \text{cap}(g), \quad (19)$$

cuyos valores para el catálogo completo del ciclo 26-Primavera (turnos matutino y vespertino) se recogen en la Tabla 1.

El mínimo de estos totales es $C_0(1113085) = 864$. Una pregunta no trivial es si las restricciones de traslape R5 reducen el máximo alcanzable por debajo de ese límite. Para responderla se enumeraron las 1,927,442 combinaciones de grupos libres de conflicto que existen en el catálogo 26-P y se resolvió el programa lineal de maximizar el número de asignaciones completas sujeto a las cotas de cupo R1. El óptimo resultó exactamente

$$\bar{N} = \min_u C_0(u) = 864; \quad (20)$$

R5 no es la restricción activa. El catálogo ofrece suficiente diversidad horaria para cubrir sin traslapes cualquier padrón $N \leq 864$, y el cuello de botella es de capacidad física, no de estructura de horarios. El padrón real de $N = 674$ queda por debajo de $\bar{N}$ con una holgura de $864 - 674 = 190$ estudiantes.

UEA	Secciones	$C_0(u)$
Introducción a la Física (1111078)	20	865
Complementos de Matemáticas (1112013)	23	965
Introducción al Cálculo (1112042)	26	1,065
Estructura Atómica y Enlace Químico (1113084)	21	910
Laboratorio de Reacciones Químicas (1113085)	24	864

Tabla 1: Capacidades brutas (19) del catálogo completo del ciclo 26-Primavera (turnos matutino y vespertino). El mínimo lo alcanza el Laboratorio de Reacciones Químicas con 24 secciones de cupo 36, lo que fija la cota del sistema.

Para que el sistema pueda atender un padrón de N estudiantes, el número de secciones del laboratorio debe satisfacer

$$n_L \geq n_L^*(N) := \left\lceil \frac{N}{36} \right\rceil, \quad (21)$$

y cada UEA académica de cupo uniforme de 45 plazas debe ofrecer al menos

$$n_A \geq n_A^*(N) := \left\lceil \frac{N}{45} \right\rceil \quad (22)$$

secciones. La Tabla 2 evalúa las condiciones (21) y (22) para los tamaños de padrón más relevantes, tomando como referencia el catálogo del ciclo 26-P ($n_L = 24$, $n_A = 20$ para la UEA académica de menor capacidad bruta, $\bar{N} = 864$). Las filas con $\Delta < 0$ señalan holgura en la oferta vigente; las filas con $\Delta > 0$ indican las secciones adicionales que la División deberá abrir.

N	$n_L^*(N)$	Δn_L	$n_A^*(N)$	Δn_A	Sin cambios a UEA académicas
600	17	-7	14	-6	✓
674	19	-5	15	-5	✓
810	23	-1	18	-2	✓
864	24	0	20	0	✓
900	25	+1	20	0	×
1,008	28	+4	23	+3	×
1,100	31	+7	25	+5	×

Tabla 2: Secciones mínimas requeridas (21)–(22) en función del padrón N . Las columnas Δ miden el incremento (+) o la holgura (–) respecto al catálogo del ciclo 26-Primavera ($n_L = 24$, $n_A = 20$ para la UEA académica más restringida, $\bar{N} = 864$). Para $N = 900$ se requiere una sección adicional de laboratorio; las UEA académicas no necesitan cambios hasta $N > 865$.

El sistema aplica prioridad matutina al alumnado femenino, reservando un *buffer masculino* de 9 plazas por sección académica de cupo 45 ($\lfloor 0.2 \times 45 \rfloor = 9$). La capacidad efectiva para alumnas en las 17 secciones matutinas de cada UEA académica es

$$C_F = 17 \times 36 = 612 \text{ plazas}, \quad (23)$$

con mínimo garantizado $C_M = 17 \times 9 = 153$ plazas para alumnos varones. El buffer es una restricción distributiva, no de capacidad total: la cota (20) permanece en $\bar{N} = 864$. En 26-Primavera, con $n_F = 186$ alumnas y $n_M = 488$ alumnos varones, la demanda femenina es muy inferior a $C_F = 612$: la cuota no está saturada y las plazas disponibles fluyen al alumnado masculino. El valor C_F funciona como umbral de alerta: si en un ciclo futuro n_F superase 612 habría que abrir secciones adicionales o revisar el criterio de prioridad.

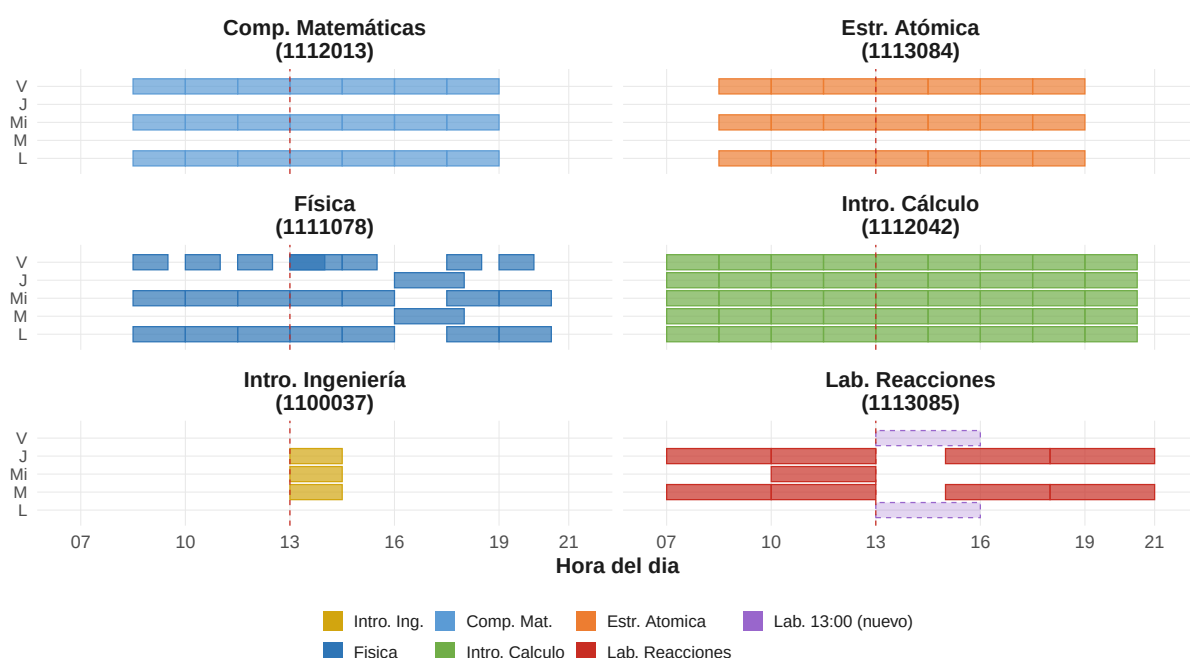
2.7 Verificación computacional y extensión al carril de 13:00

La Sección 2.6 establece la cota $\bar{N} = 864$ a partir del catálogo completo del ciclo 26-P. Esta sección presenta la verificación independiente obtenida con un modelo en R que lee directamente el archivo de horarios del Excel, y analiza en qué condiciones puede extenderse la cota abriendo secciones de laboratorio en el carril de 13:00.

Datos del catálogo. El análisis lee la hoja de horarios del ciclo 26-P y extrae, para cada UEA y grupo, los pares (día, franja) con sesión programada. La Figura 1 muestra el resultado: cada rectángulo ocupa el bloque horario de uno o más grupos; la línea vertical roja marca las 13:00. Del catálogo real se extrae que la UEA 1100037 (Introducción a la Ingeniería) tiene sesiones de 13:00 a 14:30 *únicamente* en martes (7 grupos), jueves (4 grupos) y miércoles (1 grupo); *no hay ningún grupo en lunes ni en viernes*. Por tanto, de acuerdo con la condición (18), todo nuevo bloque de laboratorio en el carril de 13:00–16:00 que caiga en lunes o en viernes tiene intersección vacía con cualquier grupo de 1100037 y es incondicionalmente compatible con R5.

Franjas horarias por UEA — catálogo 26-Primavera

Cada rectángulo ocupa el bloque horario del grupo; línea roja = 13:00; morado punteado = extensión



Morado punteado: franjas 13:00–16:00 adicionales para Lab. Reacciones (sin conflicto con 1100037)

Figura 1: Franjas horarias asignadas a cada UEA en el catálogo del ciclo 26-Primavera, organizadas en seis paneles. Cada rectángulo abarca el bloque horario del conjunto de grupos correspondiente; la línea roja señala las 13:00 horas. En el panel del Laboratorio de Reacciones Orgánicas 1113085 se distinguen los rectángulos morados con borde punteado, que corresponden a las nuevas secciones en el carril 13:00–16:00 disponibles en lunes y viernes, y cuya compatibilidad con la oferta académica queda sujeta a la condición (18) de no traslape con 1100037.

Verificación de la cota. Con los cupos nominales del catálogo —36 plazas por sección de laboratorio y 45 por sección académica— la capacidad bruta total de cada UEA es

$$C_0(1113085) = 24 \times 36 = 864, \quad (24)$$

$$C_0(1111078) = 20 \times 45 = 900. \quad (25)$$

La Figura 2 (panel izquierdo) muestra el ranking completo de capacidades brutas (19): el laboratorio es el cuello de botella con $C_0 = 864$, y Física ocupa el segundo lugar con $C_0 = 900$. La programación lineal sobre las 1,927,442 combinaciones libres de conflicto confirma que R5 no reduce esta cota: el óptimo alcanzable es exactamente $\bar{N} = 864$ (20).

Sensibilidad y extensión al carril de 13:00. El panel derecho de la Figura 2 muestra cómo evoluciona $\bar{N}$ al añadir secciones adicionales de laboratorio. Para el escenario base (Física con 20 secciones), la relación es

$$\bar{N}(n_L) = \min(36 n_L, 900), \quad (26)$$

es decir, cada nueva sección en el carril de 13:00 (lunes o viernes) aporta 36 plazas hasta que Física empata el laboratorio en $\bar{N} = 900$ con $n_L = 25$. A partir de ese punto la restricción activa pasa a ser la UEA 1111078: para alcanzar $\bar{N} = 945$ se requiere tanto una sección adicional de Física (21 secciones, $C_0 = 945$) como $n_L \geq \lceil 945/36 \rceil = 27$ secciones de laboratorio.

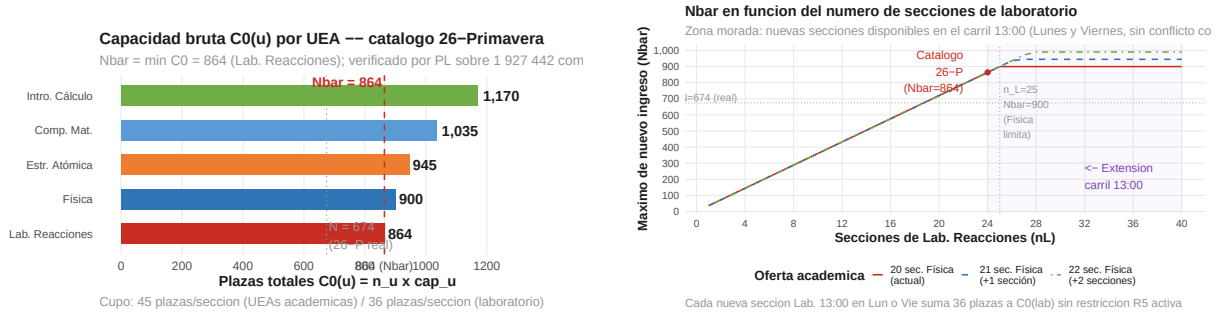


Figura 2: Panel izquierdo: capacidad bruta $C_0(u)$, definida en (19), para las cinco UEA del catálogo 26-P distintas de 1100037; la línea roja corresponde a la cota $\bar{N} = 864$ establecida en (20). Las UEA cuya barra no alcanza dicha cota representan el cuello de botella del ciclo vigente. Panel derecho: la relación de sensibilidad (26) muestra cómo varía $\bar{N}$ con el número de secciones de laboratorio n_L bajo tres escenarios de oferta académica; la franja morada delimita las secciones adicionales aportadas por el carril de 13:00 (lunes y viernes) que aparece en la Figura 1.

Condiciones de demanda. La Figura 3 representa las condiciones (21) y (22) como funciones escalón del padrón $\bar{N}$. Las líneas de puntos horizontales marcan la oferta vigente en 26-P ($n_L = 24$, $n_A = 20$ para Física); la zona dorada es el rango factible sin modificar el catálogo. Para cualquier $\bar{N}$ en ese rango, el sistema produce una asignación completa sin necesidad de nuevas secciones. La tabla 2 sintetiza los umbrales discretos más relevantes.

El modelo completo, incluyendo la lectura del Excel, el cálculo de capacidades brutas (24)–(25) y la construcción de todas las figuras de esta sección, se encuentra en el archivo `analisis_capacidad_CAT26P.R` del directorio raíz del proyecto.

3 Modelos de datos

El módulo `agents/models.py` implementa las tres estructuras de datos que circulan entre los agentes. Cada clase es la realización computacional directa de un objeto matemático de la Sección 2: `TimeSlot` representa una franja $\sigma \in \mathcal{D} \times \mathbb{T}^2$ de (2); `Group` representa un grupo $g \in \mathcal{G}$ de (1); y `Student` representa un estudiante $s \in \mathcal{S}$ de (3). Mantener esta correspondencia explícita facilita trazar cómo el código de los agentes (Sección 5) opera directamente sobre los conceptos del modelo.

No hay base de datos persistente: el estado vive íntegramente en memoria durante la ejecución. El decorador `@dataclass` (del módulo estándar `dataclasses`) genera automáticamente `__init__`,

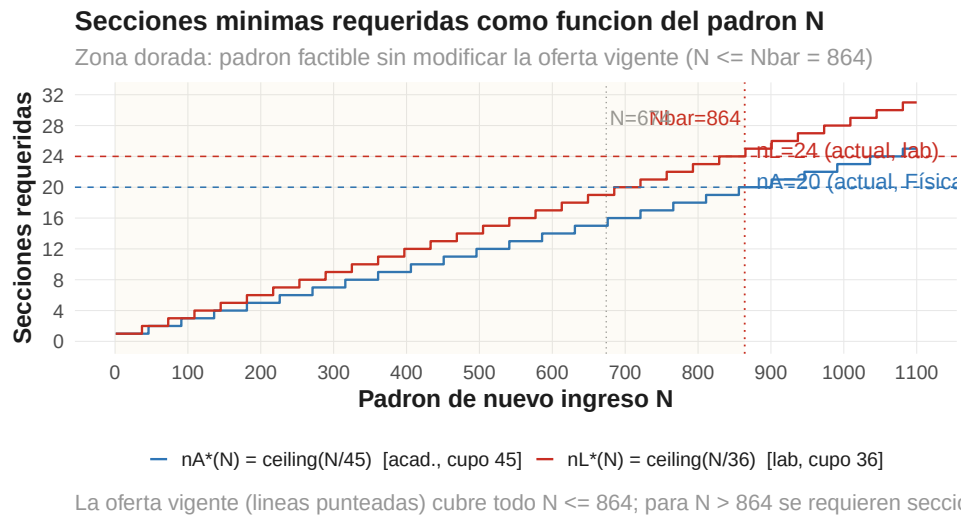


Figura 3: Secciones mínimas requeridas como funciones escalón del padrón N : la demanda de secciones de laboratorio $n_L^*(N) = \lceil N/36 \rceil$, dada por la condición (21), y la demanda académica $n_A^*(N) = \lceil N/45 \rceil$, dada por (22). Las líneas de puntos horizontales indican la oferta vigente en el ciclo 26-Primavera; la zona dorada delimita el intervalo factible $N \leq \bar{N} = 864$, cuya cota se obtiene de (20). Se observa que la demanda de laboratorio escala más rápido que la académica, de modo que es (21) la restricción efectivamente activa sobre el padrón máximo atendible.

`__repr__` y `__eq__`.

3.1 TimeSlot — franja horaria

```
@dataclass
class TimeSlot:
    day: str          # Nombre del día ('Lunes', 'Martes', ...)
    start_time: str    # Formato 'HH:MM'
    end_time: str      # Formato 'HH:MM'
```

Implementa la condición de traslape (4) —dos franjas solapan si sus intervalos se intersectan en el mismo día—:

```
def overlaps_with(self, other: "TimeSlot") -> bool:
    if self.day != other.day:
        return False
    return self.start_time < other.end_time and other.start_time <
        self.end_time
```

La comparación lexicográfica de cadenas 'HH:MM' coincide con la comparación numérica sobre $\mathbb{T}$ por la uniformidad del formato.

3.2 Group — grupo académico

```
@dataclass
class Group:
    clave: str          # Clave de UEA (ej. '1100037')
    uea: str            # Nombre descriptivo
    group_id: str       # Identificador de carril (ej. 'CAT05')
```

```

capacity:    int                # Cupo maximo: 36 o 45
enrolled:    int = 0             # Inscritos actuales
time_slots: List[TimeSlot] = field(default_factory=list)
profesor:    str = ""

```

Tres métodos concentran la lógica del modelo de grupo. `has_capacity()` devuelve True si `enrolled < capacity`, implementando la restricción de cupo R1 (7). `is_morning_shift()` devuelve True cuando todas las franjas inician antes de las 14:00 h; el Scheduler lo usa para ordenar la preferencia de grupos para el alumnado femenino. `add_student()` incrementa `enrolled` en 1 cada vez que una asignación queda registrada.

3.3 Student — estudiante

```

@dataclass
class Student:
    matricula:    str
    name:         str
    plan:         str                # Clave del plan de estudios
    gender:       str                # 'F' o 'M'
    required_UEA: List[str]          = field(default_factory=list)
    assigned_groups: List[Group]     = field(default_factory=list)
    unassigned_UEA: List[str]        = field(default_factory=list)

```

El método `has_schedule_conflict(group)` implementa R5 (13) —la prohibición de franjas solapadas entre grupos del mismo estudiante—: itera sobre el producto cartesiano $\Sigma(g') \times \Sigma(g)$ para cada grupo g' ya asignado e invoca `overlaps_with` en cada par del mismo día.

4 Librerías y dependencias

4.1 pandas

Biblioteca de análisis de datos [4, 7]. Se emplea exclusivamente para la lectura y escritura de archivos Excel y la manipulación tabular de los datos de entrada y salida.

Función / objeto	Uso en el sistema	Módulo(s)
<code>pd.read_excel(path)</code>	Carga el padrón y el catálogo de horarios en un <code>DataFrame</code> indexado por fila.	DP, RP, OV
<code>DataFrame.iterrows()</code>	Itera sobre filas como pares (índice, <code>Series</code>); accede a cada estudiante o grupo por nombre de columna.	DP, RP, OV
<code>DataFrame.columns</code>	Introspección dinámica de encabezados para detectar columnas de UEA numéricas sin configuración explícita.	DP, OV
<code>pd.isna(val)</code>	Detección de celdas vacías (<code>NaN</code> , <code>None</code> , <code>pd.NA</code>) en franjas sin horario y estudiantes sin matrícula.	DP, RP, OV
<code>DataFrame.get(col, default)</code>	Acceso seguro a columnas opcionales (<code>PROFESOR</code>) que pueden no existir en todos los catálogos.	DP
<code>DataFrame.at[idx, col] = val</code>	Escritura celda a celda al volcar asignaciones sobre el padrón original, preservando el resto de la fila intacto.	RP
<code>DataFrame.to_excel(path, index=False)</code>	Serialización del padrón modificado a <code>.xlsx</code> sin columna de índice.	RP
<code>pd.NA</code>	Valor de celda vacía compatible con <code>openpyxl</code> ; escrito en UEA sin asignación posible.	RP

DP = *DataProcessor*, *RP* = *Reporter*, *OV* = *OverlapVerifier*.

4.2 openpyxl

Librería de creación y edición de archivos `.xlsx` en Python puro. Se emplea para la generación de reportes con formato editorial (colores, bordes, anchos de columna). Se importa opcionalmente (`try/except ImportError`): si no está disponible, los reportes se producen como texto plano sin perder la funcionalidad central de asignación.

Objeto / función	Uso en el sistema
<code>Workbook()</code>	Crea un nuevo libro de trabajo Excel en memoria.
<code>workbook.active / .create_sheet(title)</code>	Acceso a la hoja activa o creación de hojas adicionales (<i>Traslapes, Diagnóstico</i>).
<code>sheet.append(row)</code>	Añade una fila de valores al final de la hoja activa.
<code>Font(bold, color)</code>	Fuente de celda: negrita y color hexadecimal para encabezados y alertas.
<code>PatternFill(fill_type, fgColor)</code>	Color de fondo de celda para distinguir filas con traslape.
<code>Alignment(horizontal, wrap_text)</code>	Alineación del texto y activación de ajuste de línea en celdas largas.
<code>Border / Side</code>	Bordes de celda: estilo (<code>'thin'</code> , <code>'medium'</code>) y color.
<code>get_column_letter(idx)</code>	Convierte índice entero a código de columna Excel ($1 \mapsto A$, $2 \mapsto B$, ...).
<code>sheet.column_dimensions[col].width</code>	Ajusta el ancho de columna para mejorar la legibilidad.
<code>workbook.save(path)</code>	Escribe el libro finalizado al sistema de archivos.

4.3 Biblioteca estándar de Python

Módulo	Elemento	Uso en el sistema
dataclasses	@dataclass	Decorador que genera automáticamente el constructor y los métodos de comparación para TimeSlot, Group y Student.
	field(default_factory=list)	Inicializa listas mutables de forma segura por instancia; evita el antipatrón de argumento mutable por defecto.
collections	defaultdict(int)	Conteo de estudiantes por plan ($n(\pi)$) en el Scheduler: inserta automáticamente 0 en la primera consulta de una clave nueva.
	defaultdict(list)	Mapa $\pi \mapsto [\text{carriles}]$ en plan_claimed_groups: la clave siempre devuelve una lista, eliminando verificaciones explícitas.
itertools	combinations(it, 2)	Genera todos los pares $\binom{ \mathcal{G}^k }{2}$ de UEA dentro de un carril CAT en ProposalVerifier y en Auditoría B de OverlapVerifier. Complejidad: $O(n^2/2)$ pares.
datetime	datetime.time	Tipo nativo de hora; usado al parsear celdas de Excel que contienen objetos de tiempo en lugar de strings.
	datetime.timedelta	Convierte deltas de tiempo (fracciones de día) que Excel almacena en celdas con formato de hora.
math	math.isnan(val)	Verificación de float NaN en celdas de Excel que pandas no convirtió a pd.NA; necesario para columnas numéricas.
time	time.time()	Medición del tiempo total de ejecución del flujo en el Coordinator.
os	os.path.exists(path)	Comprueba la existencia de archivos de entrada antes de invocar cada agente; impide errores de runtime por paths ausentes.
sys	sys.argv	Lectura del argumento de línea de comandos (identificador del trimestre) en main.py y en verify_overlaps.py.

5 Agentes del sistema

El sistema descompone la tarea en cinco fases secuenciales, cada una con una responsabilidad bien delimitada y verificable de forma independiente. La descomposición cumple dos funciones prácticas: (1) separa la validación *a priori* de la propuesta de horarios (Fase 1) de la auditoría *a posteriori* del resultado (Fase 5), creando un ciclo de retroalimentación útil tanto en el desarrollo como en la operación ordinaria; y (2) permite diagnosticar y corregir problemas en cualquier fase sin relanzar el flujo completo. El Coordinator (Fase 0) actúa como orquestador: instancia cada agente en orden e impide que un fallo en una fase diagnóstica detenga la generación del resultado.

5.1 Agente 0 — Coordinator (orquestador)

Módulo: agents/coordinator.py

5.1.1 Responsabilidad

Gestiona el ciclo de vida del flujo de trabajo: instancia cada agente, pasa los datos entre fases, mide el tiempo total de ejecución con `time.time()` y emite advertencias cuando alguna fase detecta anomalías.

5.1.2 Interfaz pública

```
Coordinator(
    asignacion_path: str, # Padron de estudiantes
    horarios_path: str, # Catalogo de horarios
    finales_path: str, # Ruta de salida de asignaciones
    semester: str # Identificador del trimestre (ej. '26P')
)
coordinator.run_workflow()
```

5.1.3 Flujo de invocación

El Coordinator ejecuta los cinco agentes en secuencia. Primero invoca `ProposalVerifier`: llama a `verify()` y, a continuación, a `generate_excel_report()`, para validar la propuesta de horarios antes de comprometer ninguna asignación. A continuación, `DataProcessor` lee los dos archivos Excel y construye en memoria las listas de objetos: `load_data()` devuelve el par (`students`, `groups`). Con esos datos, `Scheduler` ejecuta `run_assignment()` y devuelve la lista de estudiantes con sus asignaciones definidas. `Reporter` serializa el resultado mediante `generate_reports()`. Finalmente, `OverlapVerifier` ejecuta `verify()` para certificar la validez del resultado.

Advertencia El flujo no se detiene ante errores de la Fase 1 ni de la Fase 5; ambos se registran como advertencias para permitir revisar el caso sin perder los resultados intermedios.

5.2 Agente 1 — ProposalVerifier

Módulo: agents/proposal_verifier.py **Fase:** 1

5.2.1 Responsabilidad

Detecta traslapes *intra-CAT* antes de iniciar la asignación. Si el alumnado queda asignado a un único carril k para todas sus UEA, ningún par de grupos de k debe presentar franjas solapadas.

5.2.2 Funciones implementadas

Función	Descripción
<code>_to_time(val)</code>	Convierte una celda de Excel a <code>datetime.time</code> ; maneja objetos <code>time</code> , <code>timedelta</code> , <code>float</code> y strings <code>'HH:MM'</code> o <code>'HH:MM:SS'</code> .
<code>_resolve_day_cols(columns)</code>	Detecta dinámicamente los prefijos de columna de día (L, M, MI/MI, J, V) en el <code>DataFrame</code> ; normaliza las variantes de capitalización entre ciclos 26P y 26O.
<code>_build_cat_schedule(df)</code>	Agrupar todas las UEA del catálogo por número de carril CAT: $\mathcal{G}^k = \{(\text{clave}_i, \text{uea}_i, \Sigma_i)\}$.
<code>_slots_overlap(sa, sb)</code>	Comprueba la condición $\sigma_A \cap \sigma_B \neq \emptyset$ para todo $\sigma_A \in \Sigma_a$, $\sigma_B \in \Sigma_b$; devuelve la lista de colisiones con día y horarios exactos.
<code>_gather_conflicts(schedule)</code>	Itera <code>combinations(entries, 2)</code> sobre cada carril k ; acumula conflictos llamando a <code>_slots_overlap</code> .
<code>verify()</code>	Ejecuta el pipeline completo; devuelve (<code>ok: bool</code> , <code>conflicts: list</code>).
<code>generate_excel_report(output_path)</code>	Produce el reporte con <code>openpyxl</code> : hoja <i>Traslapes</i> y hoja <i>Diagnóstico</i> .

5.2.3 Algoritmo de detección intra-CAT

Sea $\mathcal{G}^k$ el conjunto de UEA en el carril k . El algoritmo evalúa todos los pares sin repetición:

```

ENTRADA: df_horarios (DataFrame del catalogo)
SALIDA:  lista de conflictos

schedule = _build_cat_schedule(df_horarios)
conflictos = []
PARA CADA carril k EN schedule:
    entries = schedule[k]           // lista de (clave, uea, slots)
    PARA CADA par (A, B) EN combinations(entries, 2):
        hits = _slots_overlap(A.slots, B.slots)
        SI hits != []:
            conflictos.append({grupo: k, uea_a: A, uea_b: B,
                               colisiones: hits})
RETORNAR (len(conflictos) == 0, conflictos)

```

El número total de pares evaluados es $\sum_k \binom{|\mathcal{G}^k|}{2} = \sum_k \frac{|\mathcal{G}^k|(|\mathcal{G}^k|-1)}{2}$.

5.2.4 Salida

`verify()` devuelve el par (`ok: bool`, `conflicts: list`), donde `ok` es `True` si y solo si no se detectó ningún traslape. El reporte detallado se escribe en `{trimestre}/Reporte_Traslapas_Propuesta.xlsx`.

5.3 Agente 2 — DataProcessor

Módulo: `agents/data_processor.py` **Fase:** 2

5.3.1 Responsabilidad

Leer los dos archivos Excel con `pd.read_excel`, normalizar el esquema y construir las listas de `Student` y `Group` en memoria.

5.3.2 Funciones implementadas

Función	Descripción
<code>load_data()</code>	Punto de entrada: invoca <code>pd.read_excel</code> para ambos archivos y llama a <code>_process_students</code> y <code>_process_groups</code> .
<code>_process_students(df)</code>	Identifica columnas de UEA por nombre numérico (<code>str.isdigit()</code> o <code>isinstance(col, float)</code>); construye un objeto <code>Student</code> por fila no vacía (<code>pd.isna</code> filtra filas sin matrícula).
<code>_process_groups(df)</code>	Aplica la regla de capacidad diferenciada; convierte celdas de hora numéricas ($10.0 \mapsto '10:00'$) y construye objetos <code>Group</code> con sus <code>TimeSlot</code> .

5.3.3 Regla de capacidad

$$\text{cap}(g) = \begin{cases} 36 & \text{si } \text{clave}(g) = 1113085 \text{ (Lab. de Reacciones Químicas),} \\ 45 & \text{en cualquier otro caso.} \end{cases} \quad (27)$$

5.3.4 Detección dinámica de columnas de UEA

En lugar de una lista fija de claves, el agente aplica el predicado:

$$\text{es_UEA}(c) = [\text{str}(c).\text{isdigit}()] \vee [\text{isinstance}(c, \text{float}) \wedge \neg \text{isnan}(c)]. \quad (28)$$

Esto garantiza robustez ante cambios en la oferta académica sin modificar el código.

5.3.5 Salida

`load_data()` $\rightarrow$ (`List[Student]`, `List[Group]`) en memoria.

5.4 Agente 3 — Scheduler (motor de asignación)

Módulo: `agents/scheduler.py` **Fase:** 3

5.4.1 Responsabilidad

Resuelve el CSP de cobertura máxima —formulado en la Sección 2— mediante un algoritmo voraz con prioridad de género y verificación exhaustiva de restricciones.

5.4.2 Estructuras de control internas

El Scheduler mantiene cuatro estructuras de control a lo largo de la asignación. `groups_by_uea` es un diccionario $\text{clave} \mapsto \text{List}[\text{Group}]$ que permite acceso directo a los grupos candidatos por UEA; se construye en $O(|\mathcal{G}|)$ al inicializar el agente. `students_per_plan` (`defaultdict(int)`) contabiliza $n(\pi)$, el número de estudiantes por plan, necesario para evaluar el umbral de la restricción de agrupamiento R4 (11)–(12). Las dos estructuras restantes gestionan el agrupamiento para la UEA 1100037: `plan_claimed_groups_1100037` (`defaultdict(list)`) mantiene el mapa $\pi \mapsto [\text{id}(g)]$ de carriles ya reclamados por cada plan, y `group_to_plan_1100037` es su inverso $\text{id}(g) \mapsto \pi$, garantizando la exclusividad $\text{plan} \leftrightarrow \text{carril}$ impuesta por R4 (11).

5.4.3 Intuición del algoritmo

El Scheduler no explora el espacio de asignaciones de forma exhaustiva. En cambio, aplica una heurística voraz: procesa a cada estudiante en un orden fijo —alumnas primero, para garantizar la preferencia matutina— y, para cada estudiante, asigna sus UEA una por una tomando el primer grupo disponible que satisface todas las restricciones activas. La clave de la eficiencia es que cada restricción se verifica de forma *incremental* sobre la asignación parcial ya construida: en el momento de asignar la UEA u al estudiante s , los grupos anteriormente asignados a s están en `assigned_groups`, de

modo que la verificación de traslapes (4) se reduce a comparar el candidato contra ese conjunto ya conocido.

El algoritmo no retrocede (*no backtracking*): si ningún grupo disponible para una UEA satisface las restricciones, esa UEA queda en `unassigned_UEA` y el proceso continúa con la siguiente. Este diseño mantiene la complejidad en $O(N)$ respecto al número de estudiantes, a costo de no garantizar el óptimo global de (14).

5.4.4 Pseudocódigo del algoritmo voraz

```

ENTRADA: S (lista de estudiantes), G (lista de grupos)
SALIDA: S con  $\phi(s,u)$  definida para todo par factible

// Pre-procesamiento
groups_by_uea <- {clave(g): [g] para g en G}
students_per_plan <- {pi(s): conteo para s en S}

// Ordenamiento por genero (alumnas primero)
S_ord <- sorted(S, key = lambda s: s.gender != 'F')

PARA CADA s EN S_ord:
  PARA CADA u EN R(s) (maximo 6):
    G_u <- groups_by_uea[u]

    // Preferencia matutina para alumnas
    SI s.gender == 'F':
      G_u <- sorted(G_u, key = not is_morning_shift)

    asignado <- False
    PARA CADA g EN G_u:
      SI NOT g.has_capacity():           continuar // R1
      SI NOT id(g).startswith('CAT'):    continuar // R2
      SI is_excluded(u, id(g)):          continuar // R3
      SI u == '1100037':
        SI plan_conflict(s, g):          continuar // R4
        SI s.has_schedule_conflict(g):   continuar // R5
      // Asignar
      phi(s, u) <- g
      g.enrolled += 1
      reclamar_carril_si_aplica(s, g)
      asignado <- True
      break

    SI NOT asignado:
      s.unassigned_UEA.append(u)

RETORNAR S_ord

```

5.4.5 Funciones implementadas

Función	Descripción
<code>__init__</code>	Construye <code>groups_by_uea</code> y <code>students_per_plan</code> ; inicializa los mapas de agrupamiento para R4 (11)–(12).
<code>_is_group_excluded(uea, group_id)</code>	Implementa R2 (8) y R3 (9): rechaza grupos no-CAT y los carriles de complemento CAT12–CAT18 para las UEA de $\mathcal{U}_{\text{exc}}$.
<code>run_assignment()</code>	Ejecuta el bucle voraz principal; delega la verificación de R5 (13) a <code>Student.has_schedule_conflict</code> .
<code>_assign(student, group)</code>	Registra $\phi(s, u)$ en <code>student.assigned_groups</code> e incrementa <code>group.enrolled</code> ; actualiza los mapas de R4 (11)–(12) cuando $u = 1100037$.

5.4.6 Complejidad

La complejidad del bucle voraz en el peor caso, derivada en la expresión (16), es $O(N \cdot U_{\text{máx}}^2 \cdot G_{\text{máx}} \cdot S_{\text{máx}})$. Para cada grupo candidato, la verificación de traslapes de la condición (4) (`has_schedule_conflict`, restricción R5 (13)) recorre los grupos ya asignados al estudiante en tiempo $O(U_{\text{máx}} \cdot S_{\text{máx}})$.

5.4.7 Salida

`run_assignment()` $\rightarrow$ `List[Student]` con `assigned_groups` y `unassigned_UEA` actualizados.

5.5 Agente 4 — Reporter

Módulo: `agents/reporter.py` **Fase:** 4

5.5.1 Responsabilidad

Exportar los resultados en formatos consumibles por el personal administrativo, preservando el formato y estructura del padrón original.

5.5.2 Funciones implementadas

Función	Descripción
<code>generate_reports(output_excel, output_txt)</code>	Punto de entrada; llama a <code>_generate_excel</code> y <code>_generate_metrics</code> .
<code>_generate_excel(path)</code>	Carga el padrón original con <code>pd.read_excel</code> , limpia las columnas de UEA a <code>pd.NA</code> , vuelca los <code>group_id</code> asignados mediante <code>DataFrame.at[idx, col]</code> y serializa con <code>to_excel</code> .
<code>_generate_metrics(path)</code>	Computa estadísticas sobre <code>Student.unassigned_UEA</code> y <code>Group.enrolled</code> ; escribe el resumen ejecutivo en texto plano.

5.5.3 Algoritmo de volcado

```
df <- pd.read_excel(asignacion_path)
uea_cols <- [c PARA c EN df.columns SI es_UEA(c)]
student_map <- {str(s.matricula): s PARA s EN students}

PARA CADA (idx, row) EN df.iterrows():
    mat <- str(row['matricula'])
```

```

SI pd.isna(row['matricula']) 0 mat NO EN student_map: continuar
s <- student_map[mat]
// Limpiar columnas de UEA
PARA CADA col EN uea_cols: df.at[idx, col] <- pd.NA
// Escribir asignaciones
PARA CADA g EN s.assigned_groups:
    col <- columna cuyo nombre coincide con clave(g)
    df.at[idx, col] <- g.group_id

df.to_excel(output_excel, index=False)

```

5.5.4 Salidas

El agente produce dos archivos en el directorio del trimestre: {trimestre}/Asignaciones_Finales.xlsx, que contiene el padrón con los carriles CAT asignados, y {trimestre}/Reporte_Metricas.txt, con las estadísticas de cobertura de la asignación.

5.6 Agente 5 — OverlapVerifier (auditoría post-asignación)

Módulo: agents/overlap_verifier.py **Fase:** 5

5.6.1 Responsabilidad

Certificar la validez del resultado final ejecutando dos auditorías independientes: verificación de traslapes por estudiante y coherencia del agrupamiento por plan de estudios.

5.6.2 Funciones implementadas

Función	Descripción
_to_time(value)	Misma lógica que en ProposalVerifier; convierte celdas de hora en objetos datetime.time.
_build_schedule_lookup(df_hor)	Construye el mapa (clave, grupo) $\mapsto [\sigma_j]$ a partir del catálogo de horarios; usado por ambas auditorías.
_slots_overlap(slots_a, slots_b)	Igual que en ProposalVerifier; devuelve pares solapados.
_uea_cols(df)	Predicado es_UEA aplicado a df.columns; identifica las columnas de asignación en el Excel final.
_gather_schedule_conflicts(df_asig, schedule)	Auditoría A: itera por estudiante sobre todos los pares de UEA asignadas y llama a _slots_overlap.
_verify_plan_grouping(df_asig, df_hor)	Auditoría B: construye los conjuntos $C(\pi)$ y verifica las tres invariantes de agrupamiento.
verify(output_path)	Ejecuta ambas auditorías; escribe el reporte de texto y devuelve True si no hay violaciones.

5.6.3 Auditoría A — Traslapes por estudiante

Sea $A(s) = \{(clave_i, g_i)\}$ el conjunto de pares (UEA, grupo) asignados al estudiante s . La auditoría certifica que todo par de UEA distintas asignadas al mismo estudiante produzca grupos mutuamente

compatibles:

$$\forall (u_i, g_i), (u_j, g_j) \in A(s), u_i \neq u_j : \text{compat}(g_i, g_j) = 1. \quad (29)$$

```
schedule <- _build_schedule_lookup(df_horarios)
PARA CADA (idx, row) EN df_asig.iterrows():
    UEA_asig <- [(col, row[col]) PARA col EN uea_cols SI row[col] NO
                es NA]
    PARA CADA par ((u_i, g_i), (u_j, g_j)) EN combinations(UEA_asig,
        2):
        slots_i <- schedule[(u_i, g_i)]
        slots_j <- schedule[(u_j, g_j)]
        hits <- _slots_overlap(slots_i, slots_j)
        SI hits != []: registrar_conflicto(row, u_i, g_i, u_j, g_j, hits
        )
```

5.6.4 Auditoría B — Agrupamiento por plan (UEA 1100037)

Para cada plan π , sea $C(\pi) = \{\phi(s, 1100037) : \pi(s) = \pi\}$ el conjunto de carriles asignados a ese plan para la UEA 1100037. La auditoría verifica tres invariantes:

La primera es el límite de carriles: $|C(\pi)| \leq \kappa(\pi)$ para todo plan π , que impide que un plan ocupe más carriles de los permitidos por su tamaño. La segunda es la exclusividad carril-plan: $|\{\pi : g \in C(\pi)\}| \leq 1$ para todo grupo g , garantizando que ningún carril sea compartido simultáneamente por dos planes distintos. La tercera es la cohesión intra-plan: todo el alumnado de π debe tener asignado un grupo dentro de $C(\pi)$, de modo que ningún estudiante quede disperso en un carril no reclamado por su plan.

Nota $\kappa(\pi)$ se calcula sobre $n(\pi) = |\{s \in \mathcal{S} : \pi(s) = \pi\}|$ en el padrón de *entrada* — no sobre el alumnado efectivamente asignado — para evitar falsos positivos cuando el cupo se agota antes de completar la asignación del plan.

5.6.5 Salida

`verify()` devuelve True si y solo si ninguna de las dos auditorías detectó violaciones. El informe completo se escribe en `{trimestre}/Reporte_Validacion_Traslapes.txt`.

6 Herramientas independientes

6.1 verify_overlaps.py

Ejecuta únicamente el `OverlapVerifier` sobre un archivo de asignaciones existente, sin correr el flujo completo. Genera además `Reporte_Traslapes_con_Correcciones.xlsx`: para cada traslape detectado, propone un grupo alternativo sin empalme; para la UEA 1100037, las correcciones se calculan a nivel de plan completo, respetando la exclusividad plan $\leftrightarrow$ carril.

```
python3 verify_overlaps.py 26P
python3 verify_overlaps.py 260
python3 verify_overlaps.py 26P 26P/Asignaciones_Finales_v2.xlsx
```

6.2 analyze_morning_capacity.py

Herramienta de análisis previo: compara la demanda de alumnas contra la oferta de grupos matutinos antes de la asignación. Cuantifica el déficit de capacidad matutina y permite ajustar la propuesta de horarios antes de comprometer el flujo.

```
python3 analyze_morning_capacity.py 26P
python3 analyze_morning_capacity.py 260
```

7 Punto de entrada — main.py

La función `find_horarios(semester)` implementa la búsqueda prioritaria del catálogo de horarios: intenta primero `{trimestre}/Propuesta horarios nuevo ingreso {trimestre}.xlsx` y, si no existe, recurre a `{trimestre}/Horarios CBI {trimestre} asignación autoratativa.xlsx`. Ambas rutas se verifican con `os.path.exists`, lo que permite que el sistema funcione tanto con la propuesta inicial como con el catálogo definitivo sin cambios en el código. El argumento de trimestre se lee de `sys.argv[1]` con valor por defecto `'26P'`.

```
python3 main.py          # trimestre por defecto: 26P
python3 main.py 26P
python3 main.py 260
```

8 Archivos de entrada y salida

8.1 Entradas requeridas

Archivo	Contenido
Asignacion_Automatica_{trim}.xlsx	Padrón: matrícula, nombre, ap_pat, ap_mat, plan, gen, columnas numéricas por UEA requerida.
Propuesta horarios ...{trim}.xlsx	Catálogo: CLAVE, UEA, GRUPO, columnas {L,M,MI,J,V}_I/_F.

8.2 Salidas producidas

Archivo	Agente	Descripción
Reporte_Traslapes_Propuesta.xlsx	ProposalVerifier	Traslapes intra-CAT en la propuesta
Asignaciones_Finales.xlsx	Reporter	Padrón con carriles CAT asignados
Reporte_Metricas.txt	Reporter	Estadísticas de asignación
Reporte_Validacion_Traslapes.txt	OverlapVerifier	Resultado de las dos auditorías

9 Incorporar un nuevo ciclo trimestral

Incorporar el ciclo 27P requiere tres pasos. Primero, crear el subdirectorio del trimestre:

```
mkdir 27P
```

Segundo, colocar en él los archivos de entrada con los nombres convenidos (ver §8.1):

```
27P/
|-- Asignacion_Automatica_27P.xlsx
'-- Propuesta horarios nuevo ingreso 27P.xlsx
```

Tercero, ejecutar el flujo completo:

```
python3 main.py 27P
```

No se requiere ningún cambio en el código mientras la estructura de columnas de los archivos Excel se mantenga igual.

10 Resumen de responsabilidades

Fase	Agente	Entrada	Salida
0	Coordinator	paths de archivos	orquestación del flujo
1	ProposalVerifier	catálogo de horarios	reporte de traslapes en propuesta
2	DataProcessor	padrón + catálogo	objetos Student, Group
3	Scheduler	Student, Group	Student con asignaciones (ϕ)
4	Reporter	Student asignados	Excel final + métricas
5	OverlapVerifier	Excel final + catálogo	reporte de auditoría

El sistema garantiza cero traslapes horarios por estudiante y satisface todas las restricciones $\mathcal{C} = \{R1, \dots, R5\}$ del CSP. El flujo completo termina en menos de un segundo para el padrón completo de nuevo ingreso.

11 Conclusiones

El sistema CAT26P aborda un problema de asignación masiva cuyo espacio de búsqueda es computacionalmente intratable por enumeración exhaustiva: con los parámetros del ciclo 26-Primavera, las $(26^6)^{674} \approx 10^{5722}$ configuraciones posibles exceden con creces cualquier capacidad de cómputo concebible. La solución adoptada — formular el problema como un CSP [5, 2] y resolverlo mediante un algoritmo voraz sin retroceso [1] — reduce la complejidad efectiva a $\sim 3.8 \times 10^6$ operaciones elementales, permitiendo la ejecución en menos de un segundo.

La formulación matemática de la Sección 2 establece con precisión cuatro conjuntos de objetos ($\mathcal{S}$, $\mathcal{U}$, $\mathcal{D}$, $\mathcal{G}$), la estructura de cada grupo (1) y cada estudiante (3), la condición de traslape horario (4), la función de asignación parcial (6), y las cinco restricciones operativas R1–R5 (7)–(13) que codifican las reglas del CAT. El objetivo de cobertura máxima (14) define sin ambigüedad el criterio de éxito: maximizar el número de pares (estudiante, UEA) efectivamente asignados bajo las restricciones activas.

La arquitectura de cinco agentes distribuye la responsabilidad de modo que cada fase es verificable de forma independiente [8]. El ProposalVerifier (Fase 1) garantiza que la propuesta de horarios no contenga conflictos intra-CAT antes de comprometer ninguna asignación. El DataProcessor (Fase 2) normaliza los datos en tiempo lineal $O(N + |\mathcal{G}|)$. El Scheduler (Fase 3) ejecuta el algoritmo voraz con prioridad matutina para alumnas, verificando R1–R5 de forma incremental sobre la asignación parcial ya construida. El Reporter (Fase 4) serializa el resultado preservando el formato del padrón original. El OverlapVerifier (Fase 5) certifica *a posteriori* que la solución satisface globalmente la compatibilidad horaria (5) y la coherencia del agrupamiento por plan. La separación entre validación *a priori* (Fase 1) y auditoría *a posteriori* (Fase 5) crea un ciclo de retroalimentación útil tanto en el desarrollo como en la operación ordinaria: un catálogo con conflictos intra-CAT se detecta antes de la asignación; una anomalía residual en el resultado se detecta después, sin necesidad de relanzar el flujo completo.

En el ciclo 26-Primavera, el sistema operó en dos fases. En la primera, el algoritmo voraz procesó los $N = 674$ estudiantes y asignó las cinco UEA del bloque de 07:00 a 10:00 a la totalidad del alumnado;

UEA 1100037 se asignó también mediante el algoritmo voraz con R1 activo ($\text{cap}=45$), lo que resultó en 431 asignaciones completas y 243 asignaciones que incluían las cinco UEA del primer bloque pero carecían de UEA 1100037, sin que ningún estudiante quedara sin ningún grupo. En la segunda fase, UEA 1100037 fue reasignada por bloque de plan a los 674 estudiantes, elevando las asignaciones completas de 431 a 674. Los 243 casos parciales se originan en la restricción de cupo de UEA 1100037 tal como la aplica el algoritmo voraz (R1 con $\text{cap}=45$): los primeros 45 estudiantes de cada plan por orden de prioridad obtienen su grupo de Introducción a la Ingeniería; los restantes quedan sin ella. La condición de no traslape (18) (Sección 2.6) establece que los grupos de UEA 1100037 (13:00–14:30) y los del Laboratorio 1113085 nunca se solapan, lo que permite reasignar 1100037 en un paso posterior mediante asignación por bloque de plan (toda la cohorte de un plan al mismo conjunto de $\kappa(\pi)$ carriles, sin imponer cupo estándar). Este paso elevó las asignaciones completas de 431 a 674, sin introducir ningún traslape, pues la condición (18) garantiza que ningún grupo de UEA 1100037 puede solaparse con grupo alguno del bloque de 07:00 a 10:00. Las auditorías del `OverlapVerifier` confirmaron cero traslapes horarios en las asignaciones producidas y coherencia plena del agrupamiento por plan para la UEA de Introducción a la Ingeniería (1100037).

El sistema es determinístico: dado el mismo padrón y el mismo catálogo, el ordenamiento fijo del algoritmo voraz garantiza la misma asignación en toda ejecución posterior. Incorporar un nuevo ciclo trimestral requiere únicamente colocar los archivos de entrada en el directorio correspondiente y ejecutar `python3 main.py {trimestre}`; no se modifica el código fuente.

El análisis de capacidad de la Sección 2.7 revela que la División puede atender un incremento de hasta $\bar{N} - N = 190$ estudiantes adicionales sin modificar el catálogo vigente. El cuello de botella es el Laboratorio de Reacciones Químicas (1113085): con 24 secciones de cupo 36, su capacidad bruta total es $C_0 = 864$ (24). La vía más inmediata para elevar ese límite es abrir secciones de laboratorio en el carril de 13:00–16:00 en lunes o viernes, días en los que el catálogo 26-P no registra ninguna sesión de 1100037: cada nueva sección suma 36 plazas a la cota (26) sin activar la restricción R5. Con una sola sección adicional ($n_L = 25$), $\bar{N}$ alcanza 900 y la restricción activa pasa a ser Física (1111078); a partir de ese punto ambas UEA deben crecer en paralelo.

Referencias

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, MA, 4 edition, 2022.
- [2] Rina Dechter. *Constraint Processing*. The Morgan Kaufmann Series in Artificial Intelligence. Morgan Kaufmann, San Francisco, CA, 2003.
- [3] Rhyd Lewis. A survey of metaheuristic-based techniques for University Timetabling problems. *OR Spectrum*, 30(1):167–190, 2008.
- [4] Wes McKinney. Data structures for statistical computing in Python. In Stéfan van der Walt and Jarrod Millman, editors, *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.
- [5] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, 4 edition, 2020.
- [6] Andrea Schaerf. A survey of automated timetabling. *Artificial Intelligence Review*, 13(2):87–127, 1999.
- [7] The pandas development team. `pandas-dev/pandas`: Pandas, 2024. Versión 2.2.3.
- [8] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, Chichester, 2 edition, 2009.